DevOps Incident Watchdog

Auto-Incident to Agile Ticket Pipeline & Continuous Delivery Architecture

Student: S Hemal	**Reg. No:** 24MIS0050	**Course:** Agile Dev Process & DevOps	**AWS Region:** ap-southeast-2

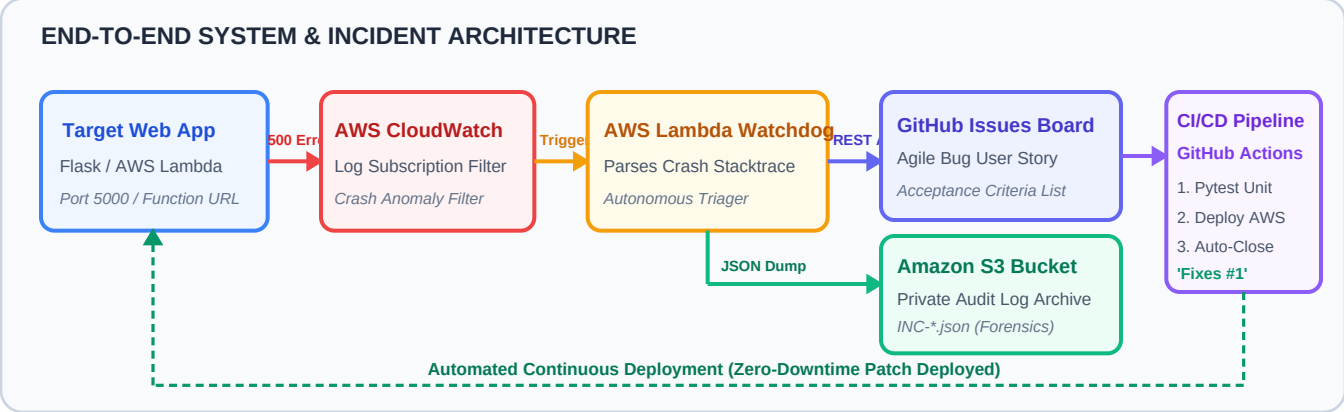
Project Essence: *An autonomous, closed-loop SRE and Agile pipeline deployed on AWS. When unexpected runtime crashes occur in production (e.g. flash sale inventory depletion causing HTTP 500 errors), AWS CloudWatch and AWS Lambda automatically capture the stack trace, archive the forensic crash dump in Amazon S3, and generate a structured Agile Bug User Story with Acceptance Criteria on the GitHub sprint backlog. Developers commit fixes with smart syntax (Fixes #1), triggering automated unit testing and zero-touch production deployment via GitHub Actions.*

1. Target Application & Incident Scenario

What application are we using? A production e-commerce retail store (**Flask Microservice** with zero-dependency WSGI serverless adapter). The catalog features high-demand electronics: Pro Laptop, Noise-Cancelling Headphones, and a **Flagship Smartphone** set up for a high-traffic flash sale.

The Failure Mechanism: Customers rapidly purchase the promotional smartphone until inventory drops to 0. The legacy checkout calculation routine attempts an unhandled order ratio calculation: discount = total / remaining_stock. When remaining_stock == 0, Python raises an unhandled **ZeroDivisionError: division by zero**, immediately triggering an **HTTP 500 Internal Server Error**. In the fixed version, inventory boundary validation returns an HTTP 409 Conflict (Sold Out) without crashing.

2. Process Architecture & Component Flow



3. Detailed End-to-End Incident Lifecycle (What All Happens)

Phase / Step	Component	Action & Technical Process	Output / Proof
Step 1: Checkout Crash	Flask App / Lambda URL	User orders last smartphone. Stock reaches 0. App raises ZeroDivisionError.	HTTP 500 Server Error Console log emitted
Step 2: Log Intercept	AWS CloudWatch	CloudWatch log filter detects pattern [INCIDENT_DETECTED] Type=ZeroDivisionError.	Gzip base64 event payload dispatched
Step 3: Watchdog Triage	AWS Lambda (Python 3.12)	Extracts error name, timestamp, log group, and 1500-character stack trace.	Generates unique Incident ID (e.g. INC-1791945923)
Step 4: Forensic Storage	Amazon S3 Bucket	Stores immutable JSON crash dump at s3://devops-...-logs/incidents/YYYY-MM-DD/.	Persistent crash audit file (369 B JSON)
Step 5: Ticket Creation	GitHub REST API v3	Lambda sends HTTP POST to /repos/.../issues with formatted User Story and Acceptance Criteria.	Agile Ticket created in GitHub Issues (e.g. Issue #1)
Step 6: CI/CD Pipeline	GitHub Actions	Developer pushes patch with Fixes #1. Runner executes 8 pytest unit tests and builds artifacts.	Green Checkmark (■ All tests passed)
Step 7: Zero-Touch Close	Git Smart Commit / AWS	Pipeline updates AWS Lambda runtime with patched code and automatically transitions Agile ticket to **Closed / Done**.	Live app healthy (200 OK) Sprint board updated

4. Agile Backlog Structure & Acceptance Criteria

Each ticket generated by the Watchdog strictly follows standard Agile User Story conventions:

Title: [INCIDENT] ZeroDivisionError: division by zero in calculation worker
Labels: bug, incident, sprint-backlog
User Story: *As an Agile Development Team member, I need to resolve runtime failure in /aws/watchdog-target-app so that high system availability is restored and users experience zero disruption.*
Acceptance Criteria (Definition of Done):
• Root Cause Analysis (RCA) completed from crash logs archived in Amazon S3.
• Reproduce bug with automated unit test in tests/test_app.py.
• Push code fix using smart commit syntax (git commit -m "Fixes #1").
• Automated CI/CD pipeline passes all tests and deploys updated application.
• Health check endpoint (/health) returns HTTP 200 OK.

5. Faculty Viva Cheat Sheet (Quick Answers for S Hemal)

Question from Professor	Recommended High-Scoring Response (What to say)
1. What is the role of Amazon S3?	*'Amazon S3 provides private, tamper-proof forensic storage for raw crash dumps and stack traces outside our application server, ensuring compliance and audit traceability.'*
2. How does GitHub get the ticket?	*'The AWS Lambda Watchdog executes upon incident detection and calls the GitHub REST API (POST /issues) using a secure token to instantly create the Agile User Story with acceptance criteria.'*
3. What is the Agile connection?	*'Instead of manual triage, runtime production defects are automatically mapped directly into sprint backlog user stories with explicit acceptance criteria (Definition of Done).'*
4. How does the ticket close automatically?	*'Through GitHub Smart Commits: our developer references Fixes #1. When GitHub Actions passes all tests and deploys, GitHub auto-closes the issue with bidirectional traceability.'*